DeckxCoin

Covenant Outputs for a Peer-to-Peer Electronic Cash System

Bitcoin's unspent-output value layer, an Ethereum-style contract layer that holds no balance, and a Lightning-style channel network — in one chain.

Version 1.0 · August 2026
github.com/xyb3rpunq/deckxcoin
xyb3rpunq.github.io/deckxcoin

genesis 000081f3be3827f4e30701ab4ed75563fb610c1735154f4b41b83b8f5c444f00
supply 21,000,000 DECKX · halving every 365 days
tests 189 passing
licence MIT

Not audited. No live network. DECKX is not available for purchase and has no price. Nothing in this document is financial, investment, tax, or legal advice.

Version 1.0 — August 2026 Reference implementation: `https://github.com/xyb3rpunq/deckxcoin` Live specification and explorer: `https://xyb3rpunq.github.io/deckxcoin/`

> **Not audited. No live network. DECKX is not available for purchase and has no price. Nothing in this document is financial, investment, tax, or legal advice.**

Abstract

Bitcoin established that a distributed ledger can be ordered without a trusted identity registry. Ethereum established that such a ledger can carry arbitrary state transition. Attempts to combine the two have consistently taken one of two paths, and both give something up. Abandoning the unspent-transaction-output model for accounts forfeits parallel validation and makes payment channels awkward to express. Keeping outputs but letting contracts custody value reconstructs the pooled-balance honeypot that has been the proximate cause of essentially every large-scale loss on account-model chains.

We describe a third construction. In DeckxCoin, contracts hold no balance. Value paid to a contract address remains an ordinary unspent output; the contract's only role is to answer whether a specific transaction may spend a specific output. We call these *covenant outputs*. The construction preserves Bitcoin's value model in full, provides Ethereum's expressive power over spending conditions, and eliminates cross-contract reentrancy as a category rather than mitigating it.

We further describe Volt, a Poon–Dryja payment channel network built on the same script types, and a monetary policy of 21,000,000 units with a 365-day halving interval — a schedule whose consequences for the security budget we state explicitly rather than obscure.

A complete implementation — roughly 5,800 lines of TypeScript, heavily commented, with a further 2,300 lines of tests — accompanies this document. The genesis block was mined, not hard-coded, and can be reproduced on commodity hardware in under a second.

---

1. Introduction

The two foundational designs in this space solve different problems, and both solve them well.

Nakamoto's construction answers the question of how a set of mutually distrusting parties agree on an ordering of events without appointing anyone to decide. The answer — make the ordering expensive to produce and cheap to verify, then follow the most expensive chain — remains the only known solution that does not smuggle an identity registry back in through the side door.

Buterin's and Wood's construction answers a different question: how a ledger can encode conditions richer than "this key signed." The answer — a deterministic virtual machine whose state is committed in every block header — is likewise sound.

The difficulty is that the two designs make incompatible assumptions about where value lives. Bitcoin locates value in discrete, individually-owned outputs. Ethereum locates it in account

balances, including the balances of contracts. A contract that can hold a balance is a contract that can be drained; a contract that cannot is, in the account model, barely a contract at all.

1.1 Prior approaches and what they cost

Accounts on a UTXO chain. Several designs bolt an account-based execution layer onto a UTXO-based settlement layer, bridging between the two. This works, but the bridge is now the most security-critical component in the system, and it reintroduces pooled balances on the execution side.

Contracts that custody outputs. Others keep UTXOs and give contracts an address that can hold them, with the contract's code deciding disbursement. This is closer, but if the contract's storage is the authoritative record of who owns what within the pool, then the pool is again a single failure domain.

Covenants without a VM. Bitcoin's own research direction — `OP_CHECKTEMPLATEVERIFY` (BIP-119), `OP_CHECKSIGFROMSTACK` (BIP-348), `OP_CAT` (BIP-347) — constrains how an output may be spent without introducing general computation. As of mid-2026 none has activated: BIP-119's proposed deployment window opened in March 2026 with effectively zero miner signalling, BIP-347 reached specification completeness in March 2026 but remains contested, and CTV+CSFS has emerged as the frontrunner pairing without a credible activation path. Bitcoin's last consensus change was Taproot in November 2021, the longest quiet period in its history.

The direction is right. The constraint is that each proposal must fit through a soft fork's backwards-compatibility keyhole, which forces a choice between expressiveness and deployability.

1.2 Our contribution

DeckxCoin asks what the covenant idea looks like when it is designed in from genesis rather than retrofitted. The answer is simpler than either alternative:

1. An output paid to a contract address is an ordinary unspent output.
2. It has no key. Its unlocking condition is the contract's approval.
3. The contract receives the spending transaction's context and returns two words: whether the spend

is approved, and which address must receive value.

4. Consensus enforces the second word.

That is the whole mechanism. Section 4 gives it precisely. Everything else in this paper is either the machinery needed to make it work or an examination of what it makes possible.

---

2. The value layer

The base layer is Bitcoin's, and we keep it substantially unmodified. Section references below are to the Bitcoin whitepaper.

2.1 Transactions

A coin is a chain of digital signatures (§2). Each input references a previous output by `(txid, index)` and carries a **BIP-340 Schnorr** signature over a digest committing to the entire transaction plus the *value and address of the output being spent*.

That last clause follows BIP-143's correction. Without it, a signing device can be induced to authorise a larger output than it was shown, because the signature does not commit to what it spends. Serialising the transaction body once rather than once per input additionally avoids the quadratic hashing behaviour of Bitcoin's original signature hash.

2.1.1 Why Schnorr, not ECDSA

Every signature on this chain is a 64-byte BIP-340 Schnorr signature over a 32-byte x-only public key. ECDSA is not supported at all — not deprecated, absent. Four reasons, in the order they mattered:

Batch verification. Schnorr signatures are linear, so `n` of them verify in roughly the cost of `n/2` individually. Initial block download is dominated by signature checking, and this is the largest available win that does not change the consensus rules.

Provable security. BIP-340 has a security proof under the discrete logarithm assumption. ECDSA's does not exist without additional assumptions, and its history of nonce-reuse catastrophes is not an accident of implementation.

Non-malleability by construction. ECDSA needs a low-`s` rule bolted on to stop `(r, s)` and `(r, n - s)` both verifying. Schnorr signatures are unique for a given key and message.

One code path. Supporting both would mean every verifier branches on a scheme selector, and every such branch is somewhere a check can be skipped. Bitcoin carries both because it had to; a chain designed today does not.

The cost is that public keys become x-only: a key commits to an x-coordinate and the y-coordinate is implicitly the even one. Wherever a *point* is needed rather than a key — Diffie-Hellman inside the onion, the revocation-key homomorphism — the implementation uses compressed 33-byte points explicitly. Conflating the two encodings is the one real hazard the scheme introduces, and it is not hypothetical: an early revision lifted an x-only key to its even-y point inside the revocation derivation, which silently broke the identity for the half of all keys whose point has odd y. The function names now state which encoding they take, the derivation rejects the wrong one outright, and a test exercises twenty independent parities.

Signing is deterministic: no auxiliary randomness is supplied, so the same key and digest always produce the same signature. BIP-340 permits this, and it is what makes the genesis block and every test vector reproducible on any machine.

Implementation is `@noble/curves`, the audited reference. Hand-rolling BIP-340 for a project that talks about security would be the wrong call, however short the specification looks. Correctness is demonstrated against the **official BIP-340 test vectors** — all fifteen, including the ten adversarial cases: a public key off the curve, an `R` with odd y, an `s` at the group order, an `x` at the field size.

2.2 Non-malleable identifiers

Transaction identifiers are computed over a serialisation that omits all witness data — public keys, signatures, cosignatures, and hash preimages. Two consequences follow.

First, no party can alter a transaction's identifier by re-encoding its signature. Bitcoin obtained this property only with segregated witness in 2017; here it holds from genesis, which is what makes a channel commitment safe to reference before it is broadcast.

Second, signing is well defined. A digest cannot depend on a field that comes into existence only after the digest has been signed. This is not a hypothetical: the first implementation of this protocol included the public key in the identifier preimage, and every signature failed verification.

2.3 Proof of work and difficulty

Blocks are ordered by proof of work (§4): double-SHA-256 over an 88-byte header, with the target encoded in Bitcoin's compact `nBits` float. Difficulty retargets every 2,016 blocks toward a 600-second block interval, clamped to a factor of four per period in either direction. Without the clamp, a single manipulated timestamp span can drive difficulty to a value the network cannot climb out of.

Fork choice is **most accumulated work**, summing `2<super>256</super> / (target + 1)` per header — not most blocks. The distinction is invisible while difficulty is constant and decisive the moment it is not.

Genesis difficulty is `0x1f00ffff`, roughly `2<super>-16</super>`, or about 65,536 expected attempts. This is deliberate: a genesis block nobody can reproduce is a genesis block nobody can audit. Any reader can re-mine it and confirm the published hash.

2.4 Merkle commitments

Transactions are committed by a Merkle root (§7), preserving Bitcoin's rule of duplicating the final element on odd levels. We reject at construction time the case where the last two leaves of a level are identical — the CVE-2012-2459 condition, which allowed two distinct blocks to share a root, one of them invalid. Bitcoin patched this by marking such blocks invalid; we refuse to build them.

Simplified payment verification (§8) is implemented and tested at every tree size from one to seventeen leaves.

2.5 Monetary policy

DeckxCoin issues **21,000,000 DECKX**, divisible to eight decimal places (the base unit is the *zap*), halving **every 365 days**. At 600-second spacing that is

```
365 x 24 x 6 = 52,560 blocks
```

The initial subsidy is not a free parameter. A geometric halving series sums to twice the product of the interval and the initial subsidy, so the cap and the interval over-determine it:

```
INITIAL_SUBSIDY = floor( 2,100,000,000,000,000 / (2 x 52,560) ) = 19,977,168,949 zaps
                = 199.77168949 DECKX
```

Summing all 64 eras gives 2,099,999,998,972,800 zaps — 20,999,999.99 DECKX, landing 0.0103 DECKX under the cap. Rounding the subsidy up to a tidier 200 DECKX would issue 21,024,000 DECKX and then require a special-cased final era to clip the excess. A discontinuity in the issuance curve is a consensus edge case that will be exercised exactly once, decades after anyone has thought about it. We prefer the ugly constant.

The cap is additionally enforced as a consensus rule rather than left as a property of the arithmetic. The check should never fire; that is precisely why it is cheap insurance.

The cost of a one-year halving, stated plainly. This schedule front-loads issuance severely. Approximately 50% of all DECKX exists after one year and 99.9% within a decade; Bitcoin reaches the equivalent point around 2140. The security budget therefore transitions from subsidy-funded to fee-funded roughly an order of magnitude faster. Any deployment of this schedule needs a functioning fee market within a few years, not a few decades. This is a real cost, and we do not claim otherwise.

Issuance terminates entirely at height 1,839,599, approximately 35 years in.

---

3. The state layer: the DVM

The Deckx Virtual Machine is a deterministic 256-bit stack machine with persistent per-contract storage, metered execution, and reverts that discard every write.

3.1 What is retained from the EVM

- **256-bit words.** Not because it is a sensible machine word, but because it makes a hash digest a

single stack item, and every meaningful contract manipulates digests.

- **Gas charged before execution of each opcode**, so an out-of-gas condition can never leave a

half-applied state write. Exhaustion consumes the full limit, which is what makes spam expensive.

- **Storage pricing** of 20,000 for a zero-to-nonzero write and 2,900 to overwrite, with a 4,800

refund on clearing, capped at 20% of gas used (EIP-3529's rule, for the same anti-gas-token reason).

- **A 24,576-byte code limit** (EIP-170), bounding worst-case validation.
- **Deterministic contract addresses** derived from `(deployer, nonce)`, so a deployer knows the

address before the transaction is mined.

- **A state root in every block header**, so a light client can be given a storage value and verify

it.

3.2 What is deliberately absent

□0□ and □1□. DeckxCoin contracts are leaves: they read chain context and their own storage, and nothing else. Cross-contract reentrancy is the single largest source of value lost on account-model chains, and the standard mitigations are disciplines that every developer must remember every time. Removing the opcode removes the bug class. The cost is composability, which for a chain whose contracts express spending conditions is a price worth paying; a chain aiming at composable finance would have to weigh it differently.

Contract balances. Discussed in §4.

□0□. State that can vanish beneath a pending transaction, with no remaining legitimate use.

Unbounded balance probing. A contract may read only its own balance.

Keccak. One hash family for the entire chain, with BIP-340-style tagged hashing (`SHA256(SHA256(tag) || SHA256(tag) || data)`) providing domain separation. Two structures can never produce the same preimage. Since the DVM is not EVM-bytecode-compatible, the tooling compatibility that Keccak would buy was never available.

A hexary Merkle-Patricia trie. The state commitment is a sorted binary trie with tagged leaves and branches. It provides the same canonical root and the same logarithmic proofs in roughly a tenth of the code — and therefore a tenth of the surface for a state-root divergence bug, which is the hardest class of consensus failure to diagnose.

3.3 Determinism

Execution is a pure function of `(code, calldata, context, storage)`. The context carries block number, block time, caller, call value, and a balance lookup; all are fixed once the block is fixed. There is no clock, no randomness, and no input or output.

The corollary is worth stating: **there is no on-chain randomness.** A contract requiring it must take it from calldata and accept that the caller chose it.

---

4. Covenant outputs

This section is the contribution.

4.1 The construction

Addresses are bech32m-encoded with a version byte. Version 0 denotes a key-hash address; **version 1 denotes a contract address**. An output paid to a version-1 address is an ordinary unspent output in every respect except its unlocking condition.

The contract account carries `code`, `storage`, `deployedAt`, and `deployer`. It carries no balance. Value at a contract address is held by the unspent-output set, exactly as for any other address, and a balance query sums those outputs.

4.2 The consensus rule

A transaction spending a covenant output is valid only if:

1. the transaction is of kind `call`;
2. its declared target equals the address that locked the output;

3. the contract executes successfully and returns a non-zero first word (*approved*);
4. if the contract returns a second word, some output of the transaction pays that address.

The caller presented to the contract is the first input unlocked by a key — the party that signed and is paying the fee — not simply the first input, which on a covenant spend is the contract's own output. Naming the wrong party here makes every identity-gated contract silently mis-authorise; it was a real defect during development, caught by the escrow tests.

4.3 Why rule 4 is load-bearing

Without it, an approval is a bearer instrument. Any observer of a valid approved transaction can rebroadcast it with the outputs redirected to themselves, and the contract's decision — which was about *whether* to release, not *to whom* — is silently repurposed.

Binding the approval to a recipient the contract itself names closes this. The test suite asserts the attack fails.

4.4 What this buys

Reentrancy is absent, not mitigated. There is no cross-contract call opcode and no pooled balance. There is nothing for a callback to re-enter and no pot to drain.

Blast radius is one output. A defective vesting contract on an account-model chain endangers every beneficiary's tokens. Here it endangers the outputs it guards, which belong to one party.

Parallel validation survives. Inputs name their own history. Ethereum is spending EIP-7928 (block-level access lists, targeted for the Glamsterdam fork in the second half of 2026) to recover this property by declaring in advance which state a block touches. A UTXO chain never surrendered it.

The covenant is expressible today. No soft fork, no activation threshold, no miner signalling.

4.5 Comparison

Property	Bitcoin (2026)	Ethereum	DeckxCoin
Time-locked vault	needs CTV — unactivated	yes, contract custodies funds	yes, native
Escrow with arbiter	multisig + off-chain protocol	yes, contract custodies funds	yes, native
Contract holds value	n/a	yes — the failure mode	**no, by construction**
Reentrancy possible	n/a	yes, mitigated by discipline	**no, structurally**
Parallel validation	yes	not without EIP-7928	yes
Composable contracts	n/a	yes	no — deliberate

The final row is the honest cost. DeckxCoin cannot express composable finance, and is not trying to.

---

5. The standard covenant library

Five contracts, each a spending condition rather than an instrument, each with a published storage layout, approval conditions, and caveats, and each tested against the real chain validator rather than a mock virtual machine.

TimeVault. Releases to one fixed beneficiary at or after a fixed height. No cancel path: once funded, the output is unreachable until the height passes, including by the depositor.

Escrow. Buyer, seller, arbiter, and a refund deadline. The buyer or arbiter may release to the seller; the seller or arbiter may refund the buyer; after the deadline, the buyer may self-refund. The arbiter is fully trusted for both outcomes — this is trust-minimised escrow, not trustless escrow.

Vesting. A cliff height, an interval, and a tranche count. Each approved spend counts as one tranche, so the covenant must be funded with one equally-sized output per tranche. There is no clawback.

MultiSig. M-of-N approval, where each owner registers approval in a separate transaction keyed by their own address word — arbitrary 256-bit storage keys make the owner's identity its own slot. Approvals accumulate and never expire, and the beneficiary is fixed at deployment: owners approve a payee, not an arbitrary spend.

AtomicSwap. A hash-time-locked covenant. The receiver claims by revealing a preimage; the sender reclaims after a timeout. Revealing the preimage publishes it to everyone, which is the point for a cross-chain swap and a privacy leak for anything else — the same property driving Lightning's migration from hash locks to point locks.

Every specification in the library carries at least two documented caveats, enforced by a test. A contract whose author has found no caveats has not looked hard enough.

---

6. Volt: the channel layer

Volt implements the Poon–Dryja construction over DeckxCoin's script types. A channel costs two on-chain transactions regardless of whether the parties exchange three payments or three million.

6.0 Encrypted transport

Node-to-node traffic is encrypted and authenticated. Plaintext frames let anyone on the path — a router, an ISP, a hosting provider — observe which transactions originate at which node, which is a deanonymisation oracle given away for free. Bitcoin closed the same hole with BIP-324.

Each side sends an ephemeral compressed public key in the clear; ECDH over those keys, expanded through HKDF-SHA256, yields one AEAD key and one length-cipher key per direction plus a session identifier. The salt binds the derivation to the network, so a testnet handshake can never produce mainnet keys. Direction is decided by sorting the two ephemeral keys, which removes a round trip. Every frame is an encrypted 3-byte length followed by ChaCha20-Poly1305 ciphertext, with the encrypted length authenticated as associated data. Keys ratchet forward every 4,096 frames.

The length is encrypted because message sizes are a fingerprint: a 1,366-byte payload is an onion, a 24-byte one is a handshake acknowledgement.

Two limitations, stated rather than implied away. There is no ElligatorSwift encoding, so the handshake bytes remain recognisable as curve points to an observer performing traffic classification — this defends confidentiality, not censorship-resistance. And the ephemeral key is not bound to any long-term identity, so an active adversary who can intercept and relay is not stopped. Bitcoin has the same property: encryption raises the cost of passive surveillance enormously and does nothing about an active attacker already on the path.

6.0.1 Watchtowers

A penalty transaction only punishes a cheat if somebody broadcasts it, which makes a channel safe only while its owner is online. A watchtower removes that liveness requirement, and the naive design removes it by learning the user's entire payment graph — a worse leak than the risk it addresses.

Instead each update registers a blob keyed by a hint: the first 16 bytes of the commitment transaction identifier are the lookup key, the last 16 bytes are the decryption key. The tower stores the first half and can neither read the blob nor tell which channel it guards. When the commitment appears on-chain the tower sees the full identifier, derives the key from the half it now knows, decrypts, and broadcasts.

The penalty's fee is fixed at the moment the blob is sealed, because the transaction is signed then. A fee spike between sealing and breach could leave the penalty unconfirmable, and an unconfirmable penalty is no penalty. The client therefore seals the same penalty at several fee levels — a ladder — and the tower starts at the cheapest rung and climbs each time a broadcast fails or a penalty goes unconfirmed. Rungs whose fee would exceed the swept value are never sealed.

Blobs are persisted. A tower that forgets them on restart protects a channel until the first reboot, which is worse than not existing, because the user believes they are covered.

Still missing: no reward mechanism, so nobody is paid to run one, and no accountability — a tower that quietly does nothing is indistinguishable from one that works, right until it matters.

6.1 Channel mechanics

Funding is a 2-of-2 output requiring both parties to sign an identical digest. Each party then holds an *asymmetric* commitment transaction in which its own output is encumbered by a 144-block relative timelock and spendable immediately by the counterparty's revocation key, while the counterparty's output is unencumbered.

The asymmetry is the entire security argument: **whoever broadcasts waits, and whoever broadcasts a revoked state loses everything.**

6.2 Revocation

Revocation requires a public key both parties can compute — so the owner can commit to it in its own output — whose private key only the counterparty can ever compute, and only after receiving the per-commitment secret.

No hash construction achieves this. Whoever can hash the inputs can produce the secret, and the owner knows its own per-commitment secret from the moment it generates it. We therefore use BOLT-3's elliptic-curve derivation unmodified:

```
h1 = H(revocationBasepoint || perCommitmentPoint)
h2 = H(perCommitmentPoint || revocationBasepoint)

revocationPubkey = revocationBasepoint*h1 + perCommitmentPoint*h2
revocationSecret = revocationBaseSecret*h1 + perCommitmentSecret*h2
```

The group homomorphism is load-bearing. A test asserts the point and scalar forms agree.

Per-commitment secrets form a hash chain of depth 4,096: revealing secret *i* lets the counterparty derive every earlier secret by hashing forward, giving constant storage. BOLT-3 uses a 48-bit indexed tree for the same property.

6.3 Multi-hop atomicity

Each hop's incoming hash-time-locked contract is bound to the same payment hash as its outgoing one, with strictly greater timelock headroom. A hop can claim its incoming contract only by revealing the preimage — which is exactly the secret its downstream neighbour needed. Either the preimage propagates the full length of the route and every hop is paid, or it never appears and every contract times out. No intermediate node can end up out of pocket, and the payer cannot pay without the payee being paid.

6.4 Onion routing

Routing uses Sphinx, per BOLT-4: a 1,366-byte packet with twenty hop slots, an ephemeral-key blinding chain, deterministic filler, and a per-hop message authentication code.

Packet size is constant regardless of route length. Each hop shifts the routing block and appends fresh keystream to the tail; without the sender's pre-computed filler, that tail would decrypt to zeroes and reveal the number of remaining hops. A test asserts that routes of 1, 2, 5, 12, and 20 hops produce exactly one distinct packet size.

We substitute SHA-256 in counter mode for ChaCha20 as the stream cipher, to keep the chain's dependency surface at a single hash family. Both are pseudorandom functions keyed by the per-hop shared secret; the security argument is unchanged.

6.5 Pathfinding

Fees are amount-dependent, so the amount a hop forwards depends on everything downstream of it. Search therefore runs backwards from the destination, where the amount is already known.

Two subtleties, both of which were defects during development: the sender pays no fee on its own outgoing channel, since a channel's fee is charged by the node forwarding out over it; and the sender's timelock is the first hop's incoming expiry, not a value derived from the sender's own channel policy, since the sender has no incoming contract.

The cost function is `fee + amount · riskFactor · timelockDelta`, pricing the opportunity cost of locked liquidity. Because capacity is public but its distribution is not, the router returns ranked

alternatives and the sender retries.

6.6 Position relative to Lightning in 2026

Taproot channels are deployed and cut roughly 20% from on-chain open and close costs. Splicing has merged into the specification and is enabled by default in Core Lightning. BOLT-12 offers are supported by Core Lightning, LDK, and Eclair, though not natively by LND. Point time-locked contracts are underway.

Volt implements the settled 2018–2024 feature line. Point locks are scaffolded — every channel already carries the adaptor point — but settlement still uses hash locks. Splicing, watchtowers, multi-part payments, and route hints are absent, and listed as absent.

---

7. Security analysis

7.1 What the design defends against

Reentrancy — structurally impossible; no call opcode, no pooled balance.

Approval replay and redirection — the beneficiary binding of §4.3.

Transaction malleability — identifiers exclude witness data.

Signature substitution across outputs — the digest commits to the spent output's value and address.

Merkle root ambiguity — the CVE-2012-2459 construction is rejected.

Difficulty manipulation — median-time-past ordering plus a factor-of-four retarget clamp.

Channel theft — revocation keys computable only by the wronged party, with a penalty sweeping both outputs.

Onion traffic analysis — constant packet size, per-hop ephemeral key rotation, payment-hash binding.

Unbounded execution — gas charged before each opcode; a loop terminates at the limit.

Jump-target confusion — jump destinations are pre-scanned, so a `0x5b` byte inside push data is never valid.

Supply inflation — the unspent-output total is audited against cumulative issuance in every scenario and in eleven tests, with the cap additionally enforced at consensus.

7.2 What it does not defend against

Nothing at the network layer. There is no peer-to-peer implementation, so eclipse attacks, transaction censorship, and propagation-based attacks are outside what the code addresses.

Reorganisation beyond fork choice. Accumulated work is tracked and the rule is defined, but there is no orphan pool and no state rollback.

Channel theft while offline. The penalty transaction is implemented and tested, but somebody must be online to broadcast it. No watchtower exists.

Quantum adversaries. secp256k1, like everyone else. Bitcoin's own migration is a multi-year programme that has not meaningfully begun.

Contract authoring error. The covenant model bounds the blast radius to the guarded outputs; it does not make a wrong contract right.

7.3 Assumptions

Honest majority of hash power. Standard hardness of the discrete logarithm on secp256k1 and of SHA-256 collision resistance. Correctness of `@noble/secp256k1`, `@noble/hashes`, and `@scure/base`, the three dependencies. Sufficient liveness for channel parties to observe and respond to a revoked broadcast within the 144-block window.

---

8. Implementation

Roughly 5,800 lines of TypeScript, running directly on Node 22.18 or later via native type stripping — no build step, no bundler. Three dependencies, all audited and minimal. The line count is high for the functionality because the source carries its reasoning inline; the comments are the design record, not decoration.

The test suite is the specification. One hundred and eighty-nine tests cover primitives, consensus, the virtual machine, the covenant library, and the channel layer. Assertions run against the real validator; where a component could be tested against a mock, it deliberately is not.

The genesis block is mined at load time rather than embedded. Two independent constructions produce identical bytes, which continuous integration verifies against the published hash on every push.

A twelve-step reference scenario exercises the whole system end to end: genesis, coinbase maturity, first spend, contract deployment, a covenant refusal followed by a covenant approval, channel funding, a routed off-chain payment, a burst of micro-payments, a penalty enforcement, a cooperative close, and a supply audit. The public explorer is generated from a real execution of this scenario; no figure on it is illustrative.

---

9. Limitations

Stated once, without hedging. There is no peer-to-peer layer, no persistence beyond an in-memory snapshot, no wallet or hierarchical key derivation, and no production miner. Batch signature verification has an interface but no batch primitive behind it. Gas reservations are not refunded — the excess becomes a miner tip, because a refund output would change the transaction shape. Volt lacks watchtowers, splicing, multi-part payments, and route hints.

None of these is hidden in the implementation, and each appears in the repository's limitations table.

---

10. Regulatory positioning

This is a research artefact, and its architecture is the substance of that claim rather than a disclaimer attached to it.

There is no live network, no token in existence, no sale of any kind, no premine or founder allocation, no treasury, no custody, and no promise of return. The genesis key derives from a seed phrase printed in the source; anyone can derive it, and it controls nothing because no network runs.

The structural point is that **contracts cannot pool third-party value**. Pooling is the feature that turns a contract into a common enterprise, and it is absent by construction rather than by policy. There is no minting primitive, no token standard, no fungible-claim abstraction, no liquidity pool, no lending or staking mechanism, no governance token, and no address that accrues protocol revenue. Covenants have no admin key, no upgrade path, and no pause switch — nobody can be compelled to act because nobody holds the power to.

The AtomicSwap covenant is dual-use, and we say so: atomic swaps are how trustless cross-chain trading works, and equally a way to avoid intermediaries who would otherwise perform screening. Anyone operating a service around it should assume money-transmission rules apply.

A fuller treatment is in `docs/COMPLIANCE.md`.

---

11. Conclusion

The disagreement between Bitcoin and Ethereum is usually framed as expressiveness versus simplicity. We think it is better framed as a question about where value lives. Once contracts are prevented from holding value — and are instead confined to answering whether a particular output may be spent by a particular transaction — most of the tension dissolves. The unspent-output model survives intact, with its parallel validation and its natural fit for payment channels. Contracts gain the full expressiveness of a general virtual machine over spending conditions. And the failure mode that has dominated losses on account-model chains has nowhere to occur.

The construction is not free. Contracts cannot compose, which forecloses composable finance entirely. A one-year halving front-loads issuance and compresses the timeline for a working fee market. Both costs are real, and we have tried to state them as plainly as the benefits.

The implementation is complete, tested, reproducible, and public.

References

1. S. Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System.* 2008.
2. G. Wood. *Ethereum: A Secure Decentralised Generalised Transaction Ledger.* 2014.
3. J. Poon, T. Dryja. *The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments.* 2016.
4. G. Danezis, I. Goldberg. *Sphinx: A Compact and Provably Secure Mix Format.* IEEE S&P, 2009.

5. Lightning Network Specifications (BOLT 1–11). `https://github.com/lightning/bolts`
6. BIP-141, BIP-143 — Segregated Witness and transaction digest. 2016.
7. BIP-340 — Schnorr signatures and tagged hashing. 2020.
8. BIP-350 — Bech32m address format. 2020.
9. BIP-119 — `OP_CHECKTEMPLATEVERIFY`. Proposed deployment window March 2026.
10. BIP-347 — `OP_CAT`. Specification complete March 2026.
11. BIP-348 — `OP_CHECKSIGFROMSTACK`.
12. EIP-170 — Contract code size limit. 2016.
13. EIP-3529 — Reduction in refunds. 2021.
14. EIP-7732 — Enshrined proposer-builder separation. Glamsterdam, H2 2026.
15. EIP-7928 — Block-level access lists. Glamsterdam, H2 2026.
16. CVE-2012-2459 — Bitcoin Merkle tree duplicate-leaf malleability.
17. Bitcoin Optech, *Covenants*. `https://bitcoinops.org/en/topics/covenants/`
18. Ethereum Foundation, *Glamsterdam*. `https://ethereum.org/roadmap/glamsterdam/`

DeckxCoin reference implementation, MIT licensed. Not audited. No live network. Not financial advice.